T3000 API & Protocol

Frequently Asked Questions · built Nov 13, 2026

Frequently Asked Questions

General

Q: What browsers are supported? A: Chrome 90+, Edge 90+, Firefox 88+, Safari 14+. IE not supported.

Q: Do I need to install anything? A: Just a modern web browser. The API server runs on localhost.

Q: Can I access remotely? A: Yes, configure firewall to allow port 3003 (UI) and 9103 (API).

Connections

Q: Device not discovered? A: Check network connectivity, firewall settings, and device IP. Try manual connection.

Q: Connection keeps dropping? A: Increase timeout settings, check network quality, verify no IP conflicts.

Data

Q: Why is data not updating? A: Verify device online, check auto-refresh enabled, try manual refresh.

Q: How long is data stored? A: 24 hours real-time, 1 year historical. Archive for longer retention.

Q: Can I export data? A: Yes, from Trend Logs page - CSV, Excel, or JSON format.

Performance

Q: Page loading slow? A: Reduce displayed points, increase refresh interval, close unused tabs.

Q: Database locked errors? A: System automatically retries up to 51 seconds. Normal during high load.

Alarms

Q: Alarms not triggering? A: Check alarm enabled, verify thresholds, test notification settings.

Q: Too many false alarms? A: Adjust thresholds, add deadband, increase time delay, improve filtering.

Troubleshooting

Q: Where are log files? A: Browser console (F12) for UI errors, API logs in api/logs/ folder.

Q: How to reset system? A: Clear browser cache, restart API server, reload page.

Q: Who to contact for help? A: Check documentation first, then contact support with error details.

Developer FAQ

Architecture

Q: What's the technology stack? A:

- **Frontend**: Vue 3 + Quasar + TypeScript
- **Backend**: Rust + Axum + SQLite
- **Protocol**: BACnet, Modbus TCP/RTU
- **Communication**: REST API, WebSocket

Q: How is the project structured? A:

```
TEXT
```

```
T3000Webview9/
├── src/              # Vue 3 frontend
│   ├── t3-vue/       # Main application
│   └── shared/       # Shared utilities
├── api/              # Rust backend
│   ├── src/          # API implementation
│   └── migration/    # Database migrations
└── docs/             # Documentation
```

API

Q: How do I authenticate API requests? A: Use JWT tokens:

TYPESCRIPT

```typescript
const response = await fetch('/api/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ username: 'admin', password: 'pass' })
});
const { token } = await response.json();

// Use token in subsequent requests
fetch('/api/devices', {
  headers: { 'Authorization': `Bearer ${token}` }
});
```

Q: What's the rate limit for API calls? A: No hard rate limit, but recommended:

- Point reads: Max 10/second per device
- WebSocket subscriptions: Max 100 points
- Batch operations: Max 1000 items

Q: How do I handle API errors? A:

TYPESCRIPT

```typescript
try {
  const response = await fetch('/api/devices/389001');

  if (!response.ok) {
    const error = await response.json();
    console.error('API error:', error.message);

    switch (response.status) {
      case 404:
        // Device not found
        break;
      case 500:
        // Server error - retry
        break;
    }
  }

  const data = await response.json();
} catch (error) {
  // Network error
  console.error('Network error:', error);
}
```

Database

Q: What database is used? A: SQLite with WAL (Write-Ahead Logging) mode for concurrent access.

Q: How do I access the database directly? A:

SHELL

```
sqlite3 api/t3000.db

## View devices
SELECT * FROM devices;

## Query trend data
SELECT * FROM trend_logs
WHERE device_id = 389001
  AND point = 'IN1'
  AND timestamp > strftime('%s', 'now', '-1 day');
```

Q: What's the database schema? A: Key tables:

- `devices` - Device information
- `points` - Point configurations
- `trend_logs` - Historical data
- `alarms` - Alarm definitions
- `schedules` - Schedule objects
- `users` - User accounts

BACnet/Modbus

Q: How do I add a custom BACnet object? A: Currently not supported via UI. Requires modifying Rust backend:

RUST

```
// In api/src/t3_device/bacnet.rs
impl BacnetDevice {
    pub fn read_custom_object(
        &self,
        object_type: ObjectType,
        instance: u32,
        property: PropertyIdentifier
    ) -> Result<Value> {
        // Implementation
    }
}
```

Q: Can I use Modbus RTU over USB? A: Yes, configure serial port in device settings:

```typescript
const device = await createDevice({
  protocol: 'modbus-rtu',
  port: '/dev/ttyUSB0',  // or COM3 on Windows
  baudrate: 19200,
  parity: 'none',
  stopbits: 1,
  slaveId: 1
});
```

Q: What BACnet services are supported? A:

- ReadProperty
- WriteProperty
- ReadPropertyMultiple
- SubscribeCOV
- I-Am/Who-Is
- DeviceCommunicationControl

WebSocket

Q: How do I subscribe to real-time updates? A:

TYPESCRIPT

```typescript
const ws = new WebSocket('ws://localhost:9103/ws');

ws.onopen = () => {
  ws.send(JSON.stringify({
    type: 'subscribe',
    deviceId: 389001,
    points: ['IN1', 'IN2', 'OUT1'],
    interval: 5000  // Update every 5 seconds
  }));
};

ws.onmessage = (event) => {
  const message = JSON.parse(event.data);

  if (message.type === 'point-update') {
    console.log(`${message.point}: ${message.value}`);
  }
};
```

Q: How many WebSocket connections can I have? A: No hard limit, but recommend:

- Max 10 connections per client
- Max 100 subscriptions per connection
- Use single connection with multiple subscriptions

Performance

Q: How do I optimize large data queries? A:

TYPESCRIPT

```typescript
// ✅ Good: Use pagination
const page1 = await fetch(
  '/api/trend-logs?device=389001&point=IN1&limit=1000&offset=0'
);
const page2 = await fetch(
  '/api/trend-logs?device=389001&point=IN1&limit=1000&offset=1000'
);

// ✅ Good: Use time-based filtering
const recent = await fetch(
  '/api/trend-logs?device=389001&start=' + (Date.now() - 86400000)
);

// ❌ Bad: Fetch everything
const all = await fetch('/api/trend-logs');
```

Q: Why is my React component re-rendering too often? A: Use memoization:

TYPESCRIPT

```typescript
import { useMemo } from 'react';

const DeviceChart = ({ deviceId, points }) => {
  const data = useMemo(
    () => processChartData(points),
    [points]  // Only recompute when points change
  );

  return <Chart data={data} />;
};
```

Debugging

Q: How do I enable debug logging? A:

```shell
## Frontend (browser console)
localStorage.setItem('debug', 't3000:*');

## Backend (environment variable)
RUST_LOG=debug cargo run
```

Q: How do I inspect WebSocket traffic? A: Use browser DevTools:

1. Open DevTools (F12)
2. Network tab
3. Filter by "WS"
4. Click connection to see messages

Q: Where are crash dumps stored? A:

- **Frontend**: Browser console errors
- **Backend**: `api/logs/error.log`
- **Panic dumps**: `api/crash-dumps/`

Deployment

Q: How do I build for production? A:

```shell
## Frontend
npm run build
## Output: dist/

## Backend
cd api
cargo build --release
## Output: target/release/t3000-api.exe
```

Q: Can I run on Linux? A: Yes, cross-compile:

```shell
rustup target add x86_64-unknown-linux-gnu
cargo build --release --target x86_64-unknown-linux-gnu
```

Q: How do I configure HTTPS? A: Use reverse proxy (nginx):

```nginx
server {
    listen 443 ssl;
    server_name t3000.example.com;

    ssl_certificate /path/to/cert.pem;
    ssl_certificate_key /path/to/key.pem;

    location / {
        proxy_pass http://localhost:3003;
    }

    location /api {
        proxy_pass http://localhost:9103;
    }

    location /ws {
        proxy_pass http://localhost:9103;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}
```

Next Steps

- Best Practices
- Troubleshooting
- Performance Tuning